debugging proof

name: ezlan sohani
assessment / task: parking lot tracker - part b
date: 2026-10-11

bug 1 - empty lot totals rong

1. what was the problem?

expected behaviour: total bays 3, occupied bays 0, free bays 3
actual behaviour: total bays 3, occupied bays 3, free bays 0

2. evidence of the bug

```
star@legacystarpad:~/Documents/Github/ezparking$ python3 parking_lot_tracker_part_b.py
choose an option: 4
total bays: 3
occupied bays: 3
free bays: 0
```

3. what caused the problem?

relevant code: if bay.isOccupied:
explanation: in python, referencing a method name without parentheses evaluates to the method itself which is truthy. it should call the method with parentheses.

4. how did you debug it?

i tested calculateTotals() with three free bays and printed type(bay.isOccupied) inside the loop.

5. what did you change?

corrected code: if bay.isOccupied():
i called the method with parentheses so it evaluates and returns false for free bays.

6. proof that the fix works

test performed: empty car park totals expected: total 3, occupied 0, free 3 , which is what happened

```
star@legacystarpad:~/Documents/Github/ezparking$ python3 parking_lot_tracker_part_b.py
choose an option: 4
total bays: 3
occupied bays: 0
free bays: 3
```

debugging proof

bug 2 - private attribute error on save

1. what was the problem?

expected behaviour: bay data written to parking_data.txt and program exits cleanly
actual behaviour: program crashed with AttributeError: 'ParkingBay' object has no attribute '__plate'

2. evidence of the bug

```
star@legacystarpad:~/Documents/Github/ezparking$ python3 parking_lot_tracker_part_b.py
choose an option: 5
Traceback (most recent call last):
  File "parking_lot_tracker_part_b.py", line 131, in saveData
    data_file.write(f"{bay.getBayNumber()}|{bay.__plate}|{bay.isOccupied()}\n")
AttributeError: 'ParkingBay' object has no attribute '_ParkingBay__plate'
```

3. what caused the problem?

relevant code: data_file.write(f"{bay.getBayNumber()}|{bay.__plate}|{bay.isOccupied()}\n")
explanation: __plate is private so external functions cannot access it directly due to encapsulation.

4. how did you debug it?

i read the traceback pointing to saveData() line 131 and checked the ParkingBay getter methods.

5. what did you change?

corrected code: data_file.write(f"{bay.getBayNumber()}|{bay.getPlate()}|{bay.isOccupied()}\n")
i used the public getter method getPlate() to safely access the plate string without violating encapsulation.

6. proof that the fix works

test performed: save data to file expected: saved and ended cleanly, this was fulfilled

```
star@legacystarpad:~/Documents/Github/ezparking$ python3 parking_lot_tracker_part_b.py
choose an option: 5
data saved
program ended
star@legacystarpad:~/Documents/Github/ezparking$ cat parking_data.txt
1||False  2||False  3||False
```

debugging proof

bug 3 - invalid plate rejection inside method

1. what was the problem?

expected behaviour: parkCar() rejects an empty plate and returns False
actual behaviour: parkCar() accepted whitespace-only strings, parking an empty car in bay 1

2. evidence of the bug

```
star@legacystarpad:~/Documents/Github/ezparking$ python3 parking_lot_tracker_part_b.py
choose an option: 1
enter bay number: 1
enter licence plate:
entry recorded
bay 1: occupied -
```

3. what caused the problem?

relevant code: self.__plate = plate; self.__occupied = True
explanation: the method did not strip and validate that the plate string was non-empty before updating state.

4. how did you debug it?

i tested parkCar(" ") in isolation and observed that the bay became marked as occupied without a valid plate.

5. what did you change?

corrected code: cleaned_plate = plate.strip(); if not cleaned_plate: return False, "invalid licence plate"
i added validation inside the method to protect the object state from being set to an invalid plate.

6. proof that the fix works

test performed: parkCar with whitespace plate expected: rejected, bay free which was fulfilled

```
star@legacystarpad:~/Documents/Github/ezparking$ python3 parking_lot_tracker_part_b.py
choose an option: 1
enter bay number: 1
enter licence plate:
invalid licence plate
```

final debugging reflection

which bug was the most difficult to solve, and why?
the method parentheses bug was most difficult because python did not raise an error and silently evaluated true.

what debugging technique was most useful?
testing boundary states like an empty car park and printing variable types was most useful.

what did you learn that could help you avoid or identify similar bugs in future?
i learned to always use parentheses when invoking methods and to access private attributes via public getters.